



**ECOLE MAROCAINE DES
SCIENCES DE L'INGENIEUR**
Membre de
HONORIS UNITED UNIVERSITIES



EMSI RABAT

Filière : 3^{ème} année — Ingénierie Informatique et Réseaux

Rapport de fin d'année

Domaine : Génie Logiciel — Conception & Développement d'Applications (Merise/UML)

Système de Gestion des Emprunts de Livres d'une Bibliothèque

*Conception et réalisation d'une application de gestion
appliquant la méthodologie Merise/UML*

Réalisé par :

**HOUMMANE Mohamed
BENHNICH Safouane**

Encadré par :

Mouhcine Elhassani

Année universitaire 2025–2026

Remerciements

Au terme de ce travail, nous tenons à exprimer notre profonde gratitude à toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce projet.

Nous adressons nos remerciements les plus sincères à notre encadrant, M. Mouhcine Elhasani, pour la qualité de son encadrement, sa disponibilité, ses conseils avisés et la confiance qu'il nous a accordée tout au long de ce projet.

Nous remercions également l'**EMSI Rabat** ainsi que l'ensemble du corps professoral pour la qualité de la formation qui nous est dispensée et pour les moyens mis à notre disposition durant notre parcours.

Enfin, nous exprimons notre reconnaissance à nos familles et à nos camarades pour leur soutien constant et leurs encouragements, qui nous ont accompagnés tout au long de ce travail.

Résumé

Ce rapport présente la conception et la réalisation d’une application de bureau dédiée à la gestion des emprunts de livres d’une bibliothèque. Le contexte est celui d’un établissement souhaitant fiabiliser le suivi de son fonds documentaire : prêts, retours, réservations, localisation physique des exemplaires et sanctions en cas de retard, de perte ou de vol. L’objectif est de fournir un outil simple, autonome et fonctionnel, appliquant des règles de gestion précises (quotas, durées, pénalités). La démarche de conception s’appuie sur la méthodologie **Merise/UML** : diagrammes de cas d’utilisation, modèle conceptuel de données, diagramme de classes (puis modèle relationnel) et diagrammes de séquence. La solution est implémentée en **Python** à l’aide de la seule bibliothèque standard — **SQLite** pour la persistance des données et **Tkinter** pour l’interface graphique —, ce qui garantit un déploiement sans dépendance externe. Les résultats couvrent l’ensemble des fonctionnalités visées : authentification par rôle, gestion du catalogue et des adhérents, cycle complet d’emprunt et de retour avec calcul automatique des pénalités, réservations avec file d’attente et notifications, ainsi qu’un tableau de bord d’aide à la décision.

Mots-clés : gestion de bibliothèque, emprunts, Merise, UML, Python, SQLite.

Abstract

This report presents the design and implementation of a desktop application dedicated to managing book lending in a library. The context is that of an institution wishing to make the monitoring of its collection more reliable : loans, returns, reservations, the physical location of copies, and penalties in the event of late return, loss or theft. The objective is to provide a simple, standalone and functional tool that enforces precise business rules (quotas, durations, penalties). The design approach relies on the **Merise/UML** methodology : use case diagrams, conceptual data model, class diagram (then the relational model) and sequence diagrams. The solution is implemented in **Python** using only the standard library — **SQLite** for data persistence and **Tkinter** for the graphical interface —, which guarantees deployment without any external dependency. The results cover all the targeted features : role-based authentication, catalogue and member management, a complete lending and return cycle with automatic penalty calculation, reservations with a waiting queue and notifications, as well as a decision-support dashboard.

Keywords : library management, book lending, Merise, UML, Python, SQLite.

Table des abréviations

Abréviation	Signification
UML	Unified Modeling Language
MCD	Modèle Conceptuel de Données
MLD	Modèle Logique de Données
CRUD	Create, Read, Update, Delete
IHM	Interface Homme-Machine
RG	Règle de Gestion
ISBN	International Standard Book Number
CIN	Carte d'Identité Nationale
DH	Dirham (monnaie)
SQL	Structured Query Language

Table des matières

Remerciements	i
Résumé	ii
Abstract	iii
Table des abréviations	iv
1 Introduction générale	1
1.1 Contexte	1
1.2 Problématique	1
1.3 Objectifs	1
1.4 Organisation du rapport	1
2 Étude et analyse des besoins	3
2.1 Identification des acteurs	3
2.2 Besoins fonctionnels	3
2.3 Besoins non fonctionnels	3
2.4 Règles de gestion	3
3 Conception (Merise/UML)	5
3.1 Diagramme de cas d'utilisation	5
3.2 Diagramme de classes	6
3.2.1 Du MCD au MLD	8
3.3 Diagrammes de séquence	9
3.3.1 Emprunter un livre	9
3.3.2 Retourner un livre	9
3.3.3 Réserver un livre	9
4 Réalisation	13
4.1 Choix technologiques	13
4.2 Architecture du code	13
4.3 Extraits de code clés	13
4.4 Captures d'écran de l'application	14
4.4.1 Accueil et authentification	15
4.4.2 Espace Administrateur	16
4.4.3 Espace Bibliothécaire	20
4.4.4 Espace Adhérent	26
5 Conclusion et perspectives	33
5.1 Bilan	33
5.2 Perspectives	33
Bibliographie	34

Webographie

35

1. Introduction générale

Dans ce chapitre introductif, nous présentons le contexte général du projet, la problématique à laquelle il répond, les objectifs que nous nous sommes fixés ainsi que l'organisation du présent rapport.

1.1 Contexte

La bibliothèque est un service essentiel au sein d'un établissement d'enseignement : elle met à la disposition des étudiants et des enseignants un fonds documentaire dont la circulation doit être maîtrisée. Gérée manuellement, cette circulation devient rapidement source d'erreurs : exemplaires égarés, retards non suivis, dépassement du nombre d'ouvrages autorisés par personne, ou encore difficulté à localiser physiquement un livre sur les rayonnages. L'informatisation de ces tâches répétitives permet de fiabiliser le suivi, de gagner du temps et d'offrir un service plus transparent aux adhérents.

1.2 Problématique

Le présent projet répond à la question suivante :

« Comment informatiser la gestion des emprunts d'une bibliothèque tout en assurant la traçabilité de l'emplacement physique des livres, la détection des vols/pertes, et le contrôle du nombre d'emprunts par adhérent ? »

Trois exigences se dégagent : la **traçabilité** (savoir où se trouve chaque exemplaire), la **sûreté** (détecter et sanctionner pertes et vols) et le **contrôle** (faire respecter les quotas et les délais).

1.3 Objectifs

L'objectif est de concevoir puis de réaliser une application de bureau **simple, fonctionnelle et autonome** permettant de :

- gérer le catalogue des livres et leurs exemplaires physiques ;
- gérer les adhérents et leurs droits selon leur profil ;
- enregistrer les emprunts, retours, prolongations et réservations ;
- localiser chaque exemplaire grâce à un emplacement unique ;
- appliquer automatiquement les règles de gestion (quota, durée, pénalités) ;
- offrir un tableau de bord d'aide à la décision.

La démarche s'appuie sur la méthodologie **Merise/UML** pour la phase de conception, puis sur une implémentation en **Python** reposant uniquement sur la bibliothèque standard.

1.4 Organisation du rapport

Le rapport est structuré en cinq parties : après cette introduction, le chapitre 2 analyse les besoins ; le chapitre 3 présente la conception (cas d'utilisation, classes, séquences) ; le chapitre 4 décrit la réalisation et illustre l'application par des captures d'écran ; le chapitre 5 dresse le bilan et propose des perspectives.

En conclusion, nous avons situé le cadre et les enjeux du projet ; nous abordons à présent l'analyse détaillée des besoins.

2. Étude et analyse des besoins

Dans ce chapitre, nous analysons les besoins du système : nous identifions les acteurs, recensons les besoins fonctionnels et non fonctionnels, puis formalisons les règles de gestion qui encadrent le domaine.

2.1 Identification des acteurs

Le système distingue trois acteurs, identifiés par un compte et un rôle :

Administrateur Responsable du paramétrage : gestion des rôles utilisateurs, configuration des quotas et des paramètres de gestion, gestion des emplacements physiques et consultation du tableau de bord.

Bibliothécaire Acteur opérationnel : gestion du catalogue et des exemplaires, gestion des adhérents, enregistrement des emprunts et des retours, prolongations, déclaration des vols/pertes, suivi et encaissement des pénalités.

Adhérent Usager final : consultation du catalogue, réservation d'un livre, consultation de son historique d'emprunts et de pénalités, réception des notifications de disponibilité.

2.2 Besoins fonctionnels

L'application doit couvrir les fonctionnalités suivantes :

- **Authentification** par e-mail et mot de passe, avec orientation selon le rôle ;
- **Gestion du catalogue** : création, modification, suppression des livres et de leurs exemplaires ;
- **Gestion des adhérents** : profils, statuts et quotas ;
- **Emprunts et retours** : création d'un emprunt, retour, calcul du retard, prolongation ;
- **Traçabilité de l'emplacement** : association de chaque exemplaire à un emplacement unique et mise à jour au retour ;
- **Vols et pertes** : déclaration, pénalité financière et blocage du compte ;
- **Pénalités** : calcul, suivi du paiement, exonération et déblocage ;
- **Quota d'emprunts** : contrôle du nombre d'ouvrages empruntés simultanément ;
- **Réservations et notifications** : file d'attente et avis de disponibilité ;
- **Tableau de bord** : indicateurs (livres les plus empruntés, retards, pertes/vols).

2.3 Besoins non fonctionnels

Au-delà des fonctions, le système doit être : **simple** d'utilisation (interface épurée en français) ; **portable** et **autonome** (aucune dépendance externe à installer) ; **fiable** (application systématique des règles de gestion) et **maintenable** (code structuré en modules).

2.4 Règles de gestion

Les règles métier formalisent les contraintes du domaine. Elles sont rassemblées dans le tableau 2.1 et implémentées dans le module de logique de l'application.

TABLE 2.1 – Règles de gestion appliquées par le système.

Code	Description
RG01	Quota d’emprunts simultanés : 3 ouvrages pour un étudiant, 5 pour un enseignant. La valeur est lue dans le champ <code>max_emprunts</code> de l’adhérent et reste configurable.
RG02	Durée d’emprunt de 14 jours, prolongeable <i>une seule fois</i> de 7 jours supplémentaires.
RG03	Retard facturé 2 DH par jour. Au-delà de 30 jours de retard, l’ouvrage est présumé <i>Perdu</i> .
RG04	Éligibilité du compte : tout emprunt est refusé si le compte est <i>Suspendu</i> ou <i>Bloqué</i> , ou s’il subsiste une pénalité impayée.
RG05	Emplacement unique : chaque exemplaire est rangé à un emplacement physique identifié (salle, étagère, rangée, position), garantissant sa traçabilité.
RG06	Disponibilité : un livre n’est empruntable que s’il reste au moins un exemplaire disponible.
Perte / Vol	La pénalité est égale au prix du livre augmenté de frais administratifs ; le compte reste <i>bloqué</i> tant que la somme n’est pas réglée.

En conclusion, nous avons délimité le périmètre fonctionnel et les contraintes métier du système ; ces éléments servent de fondement à la phase de conception présentée dans le chapitre suivant.

3. Conception (Merise/UML)

Dans ce chapitre, nous traduisons les besoins en modèles de conception selon la démarche Merise/UML : nous présentons successivement le diagramme de cas d'utilisation, le modèle conceptuel de données, le diagramme de classes (puis son passage au modèle relationnel) et les diagrammes de séquence des opérations principales.

3.1 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation (Figure 3.1) synthétise les interactions entre les trois acteurs et le système. Les cas communs comme *S'authentifier* sont partagés, tandis que des relations « include » factorisent les contrôles : l'emprunt *inclut* la vérification du quota et de la disponibilité, et le retour *inclut* la mise à jour de l'emplacement. Une relation « extend » relie la *notification de disponibilité* à la *réservation*, le déclenchement étant conditionné par le retour d'un exemplaire attendu.

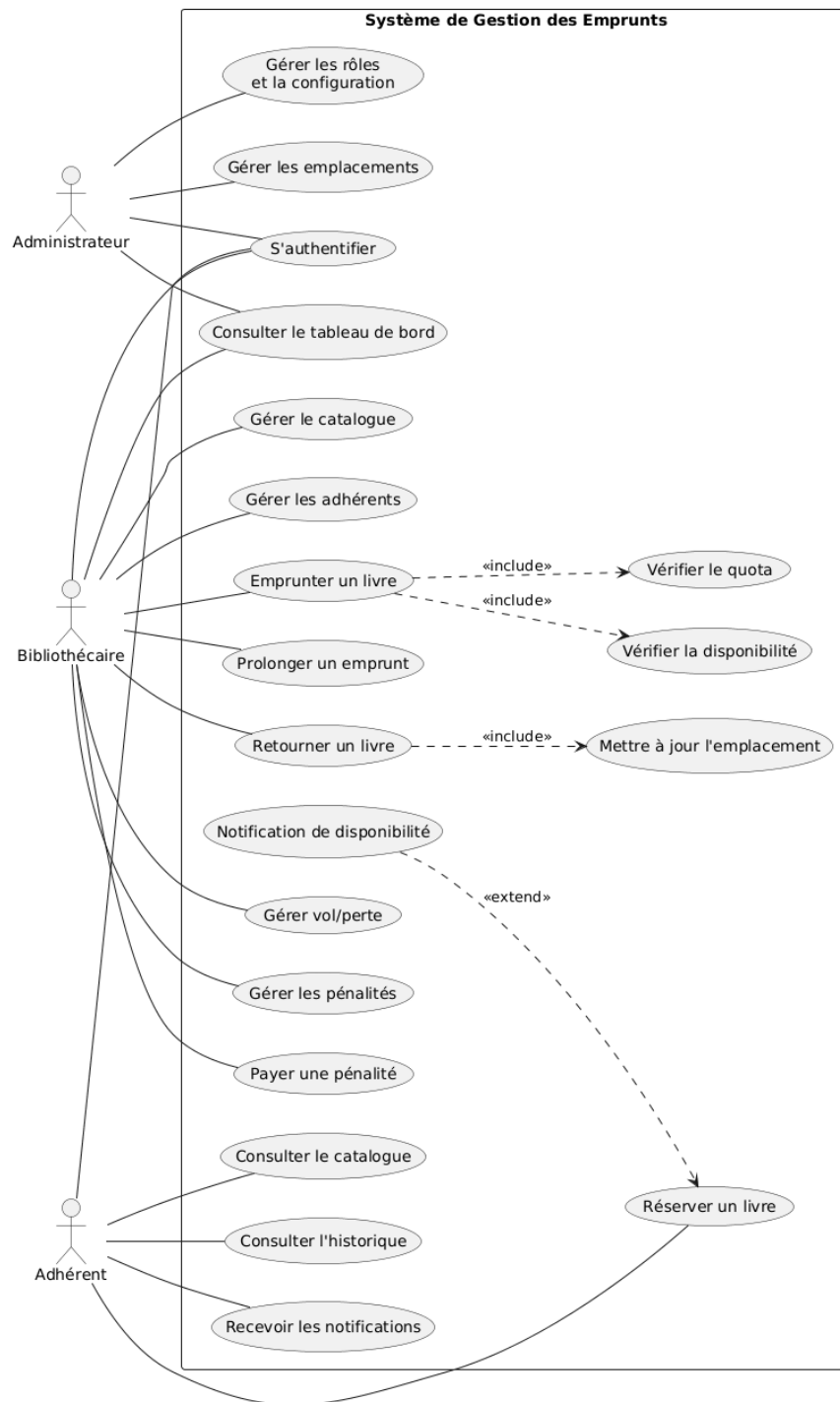


FIGURE 3.1 – Diagramme de cas d'utilisation

Acteurs (Administrateur, Bibliothécaire, Adhérent) et leurs cas d'utilisation, reliés par les relations include et extend.

3.2 Diagramme de classes

Le diagramme de classes (Figure 3.2) décrit la structure statique du domaine.

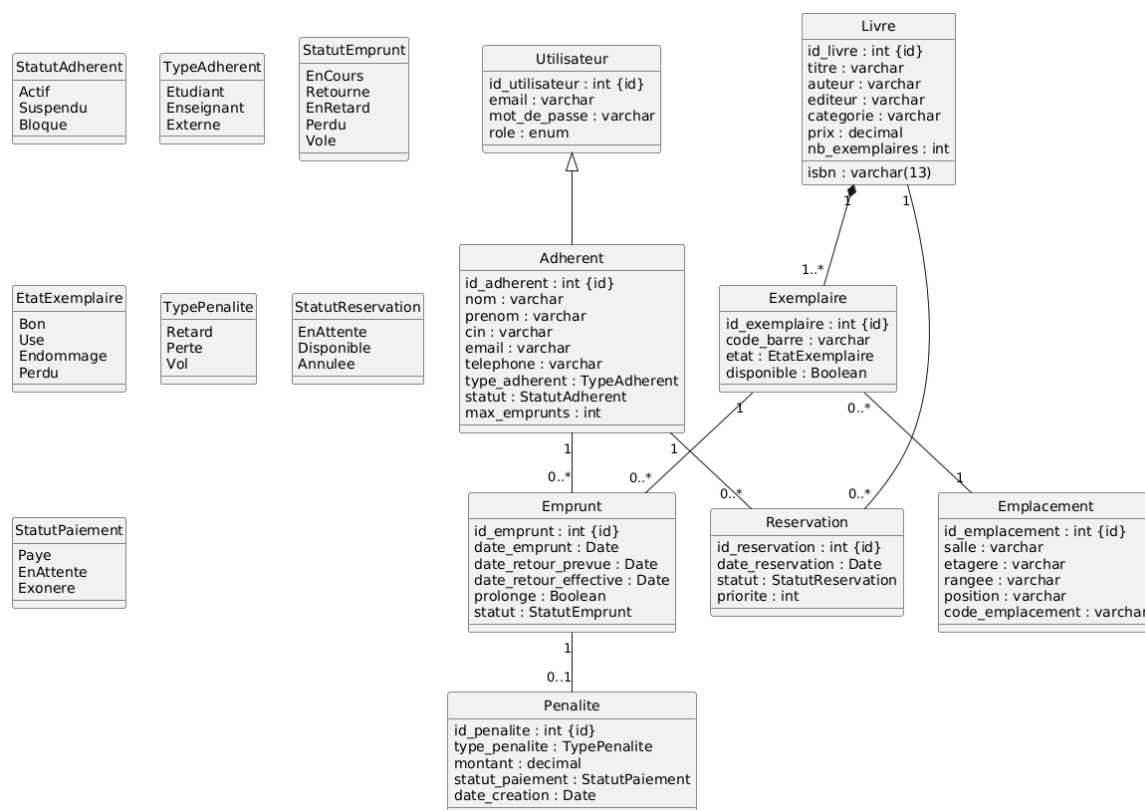


FIGURE 3.2 – Diagramme de classes

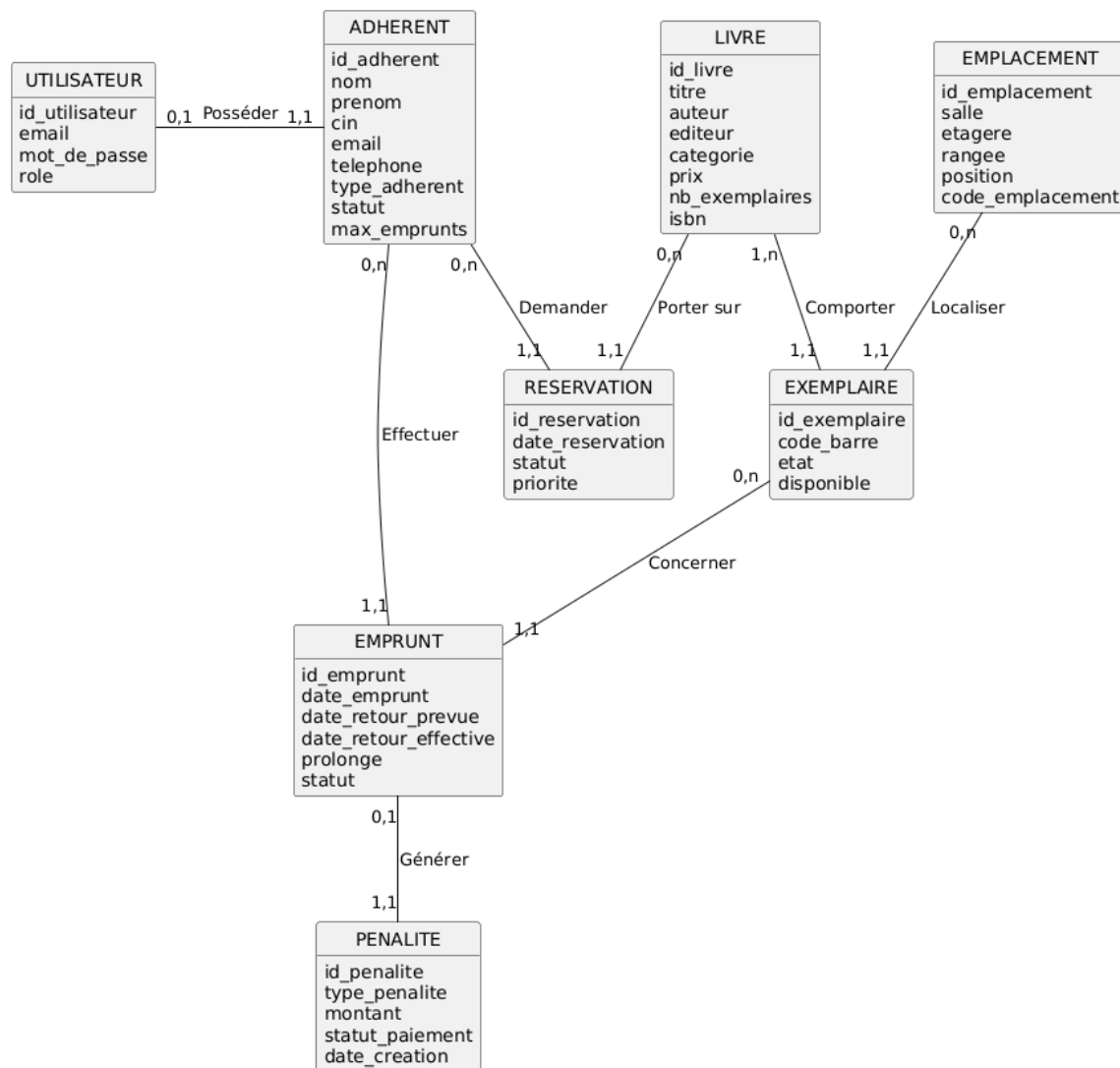
Entités du domaine, leurs attributs, les associations et les multiplicités qui les relient.

Les principales classes sont :

- **Utilisateur** : compte d'accès (e-mail, mot de passe, rôle), dont **Adherent** est une spécialisation (généralisation UML) ;
- **Livre** et **Exempleire** : un livre possède de 1 à plusieurs exemplaires physiques (composition), chacun rangé à un **Emplacement** unique (RG05) ;
- **Emprunt** : relie un adhérent et un exemplaire, avec dates et statut ;
- **Reservation** : place un adhérent en file d'attente sur un livre indisponible ;
- **Penalite** : sanction financière rattachée à un emprunt.

Les énumérations (*StatutAdherent*, *TypeAdherent*, *StatutEmprunt*, *EtatExempleire*, *TypePenalite*, *StatutReservation*, *StatutPaiement*) contraignent les valeurs autorisées.

Le **Modèle Conceptuel de Données** (MCD) ci-dessous (Figure 3.3) modélise les entités du domaine et leurs associations selon le formalisme Merise, indépendamment de toute implémentation technique.

**FIGURE 3.3 – Modèle Conceptuel de Données (MCD)**

Entités, associations et cardinalités (notation Merise) servant de base au modèle relationnel.

3.2.1 Du MCD au MLD

Le passage au **Modèle Logique de Données** traduit chaque entité en table et chaque association en clé étrangère. Les clés primaires sont soulignées et les clés étrangères préfixées par le symbole # :

- utilisateur(id_utilisateur, email, mot_de_passe, role)
- adherent(id_adherent, #id_utilisateur, nom, prenom, cin, email, telephone, type_adherent, statut, max_emprunts)
- livre(id_livre, titre, auteur, editeur, categorie, prix, nb_exemplaires, isbn)
- emplacement(id_emplacement, salle, etagere, rangee, position, code_emplacement)
- exemplaire(id_exemplaire, #id_livre, #id_emplacement, code_barre, etat, disponible)
- emprunt(id_emprunt, #id_adherent, #id_exemplaire, date_emprunt, date_retour_prevue, date_retour_effective, prolonge, statut)
- reservation(id_reservation, #id_adherent, #id_livre, date_reservation, statut, priorite)

```
— penalite(id_penalite, #id_emprunt, type_penalite, montant, statut_paiement,  
           date_creation)
```

Ce modèle est implémenté tel quel dans la base SQLite du projet.

3.3 Diagrammes de séquence

Les diagrammes de séquence détaillent la dynamique des trois opérations clés. Pour en préserver la lisibilité, chacun est présenté en pleine page, en orientation paysage.

3.3.1 Emprunter un livre

La Figure 3.4 illustre le scénario d’emprunt. Le bibliothécaire saisit l’emprunt via l’interface (IHM), qui délègue à la couche *GestionEmprunt*. Trois fragments alternatifs (`alt`) matérialisent les contrôles successifs : éligibilité du compte (RG04), respect du quota (RG01) puis disponibilité de l’exemplaire (RG06). Si tous sont satisfaits, l’emprunt est créé avec une date de retour fixée à +14 jours et l’exemplaire passe à l’état indisponible.

3.3.2 Retourner un livre

La Figure 3.5 décrit le retour. Le système récupère l’emprunt en cours, enregistre la date de retour effective et calcule le retard. Un fragment optionnel (`opt`) crée une pénalité de retard si le délai est dépassé (RG03). L’exemplaire est ensuite remis disponible, son emplacement est mis à jour, et le statut de l’emprunt devient *Retourné*.

3.3.3 Réserver un livre

La Figure 3.6 présente la réservation. Si un exemplaire est disponible, le système informe que l’emprunt direct est possible ; sinon il crée une réservation *EnAttente* et calcule la priorité (position dans la file). Le second fragment montre la *notification de disponibilité* : au retour d’un exemplaire, la réservation prioritaire passe à l’état *Disponible* et l’adhérent est notifié.

En conclusion, nous avons modélisé la structure et le comportement du système ; nous détaillons dans le chapitre suivant sa réalisation concrète.

Diagramme de séquence — Emprunter un livre

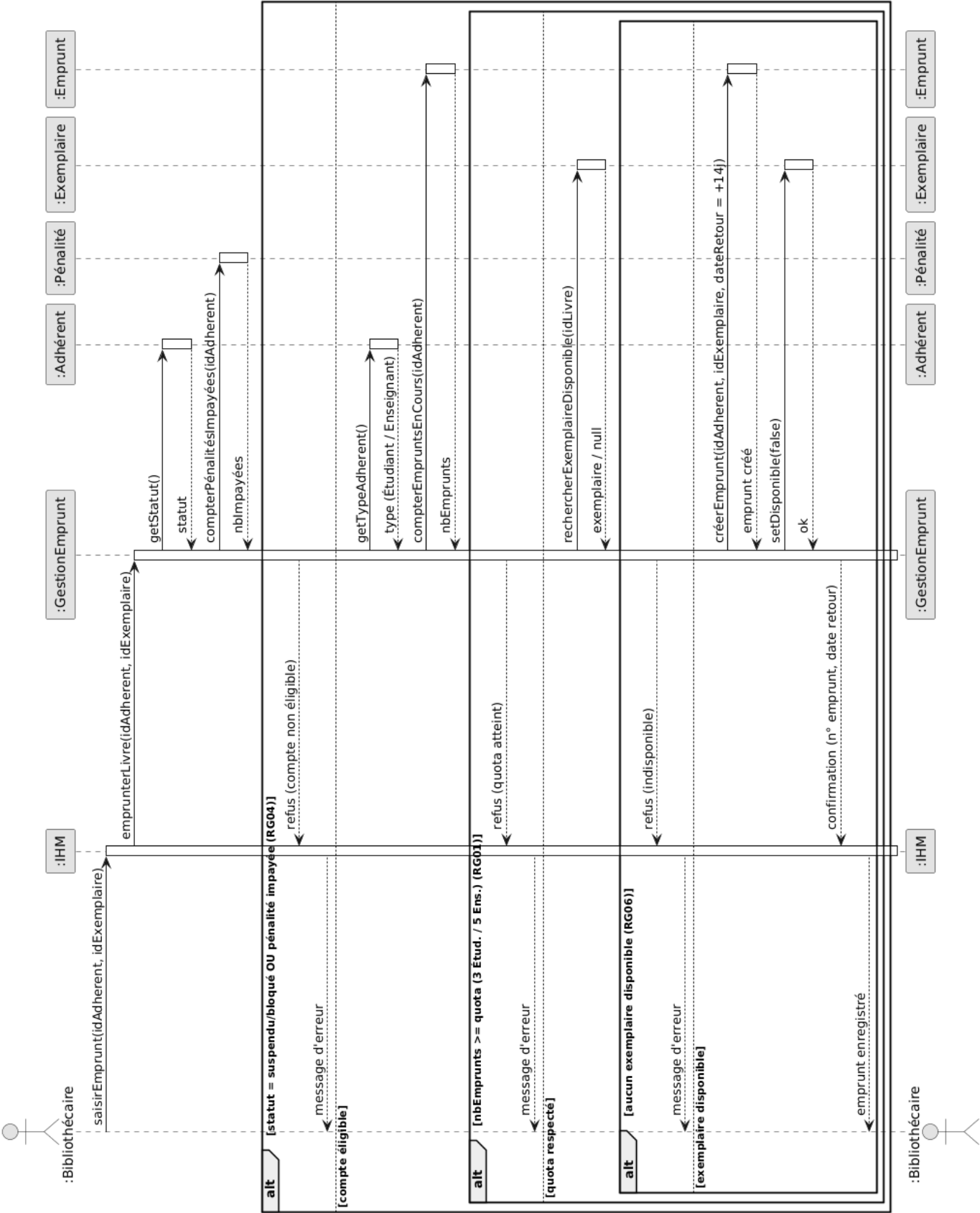


FIGURE 3.4 – Diagramme de séquence — Emprunter un livre

Enchaînement des contrôles RG04, RG01 puis RG06 avant la création de l'emprunt.

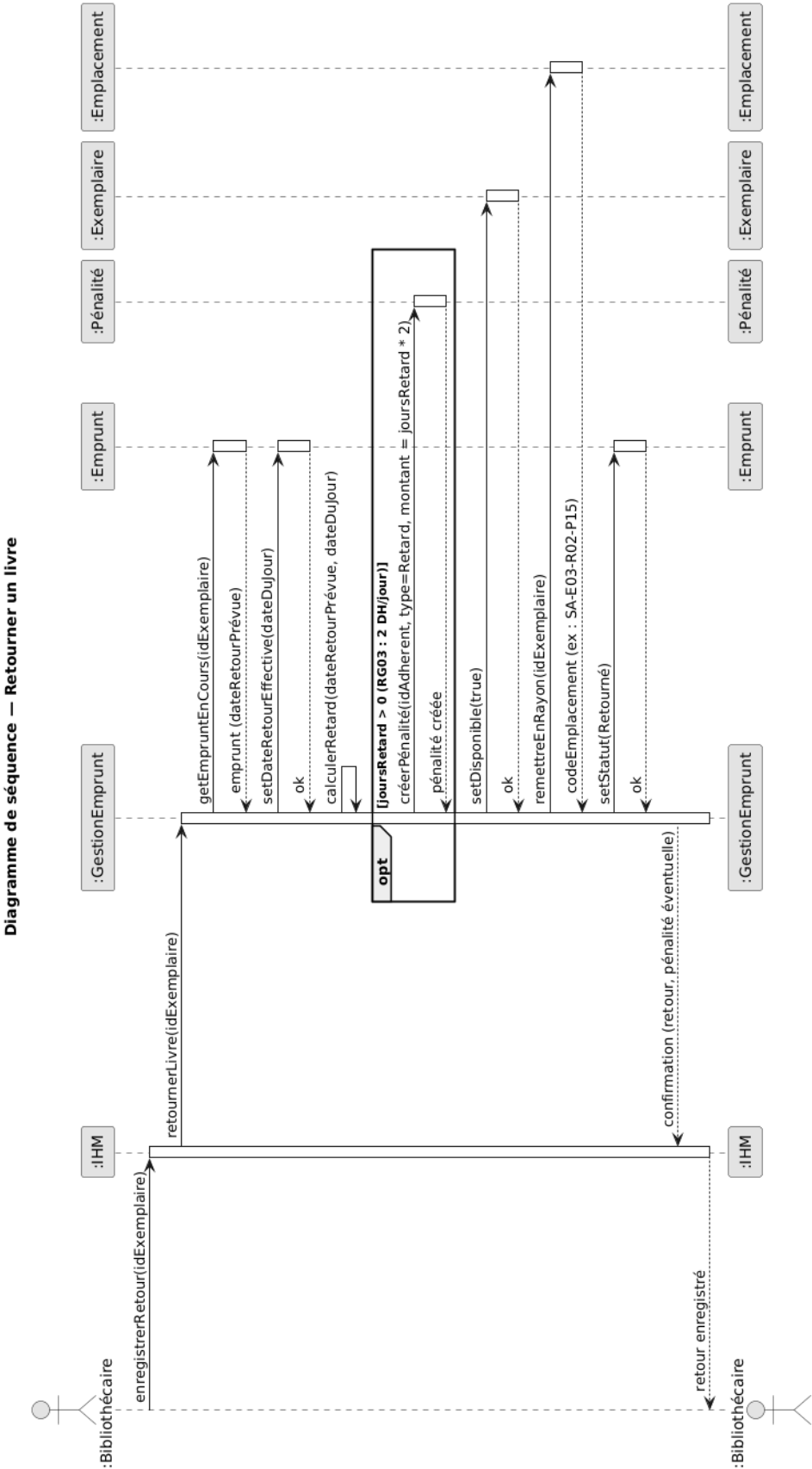


FIGURE 3.5 – Diagramme de séquence — Retourner un livre

Enregistrement du retour, calcul du retard, pénalité éventuelle (RG03) et remise en rayon.

Diagramme de séquence — Réserver un livre

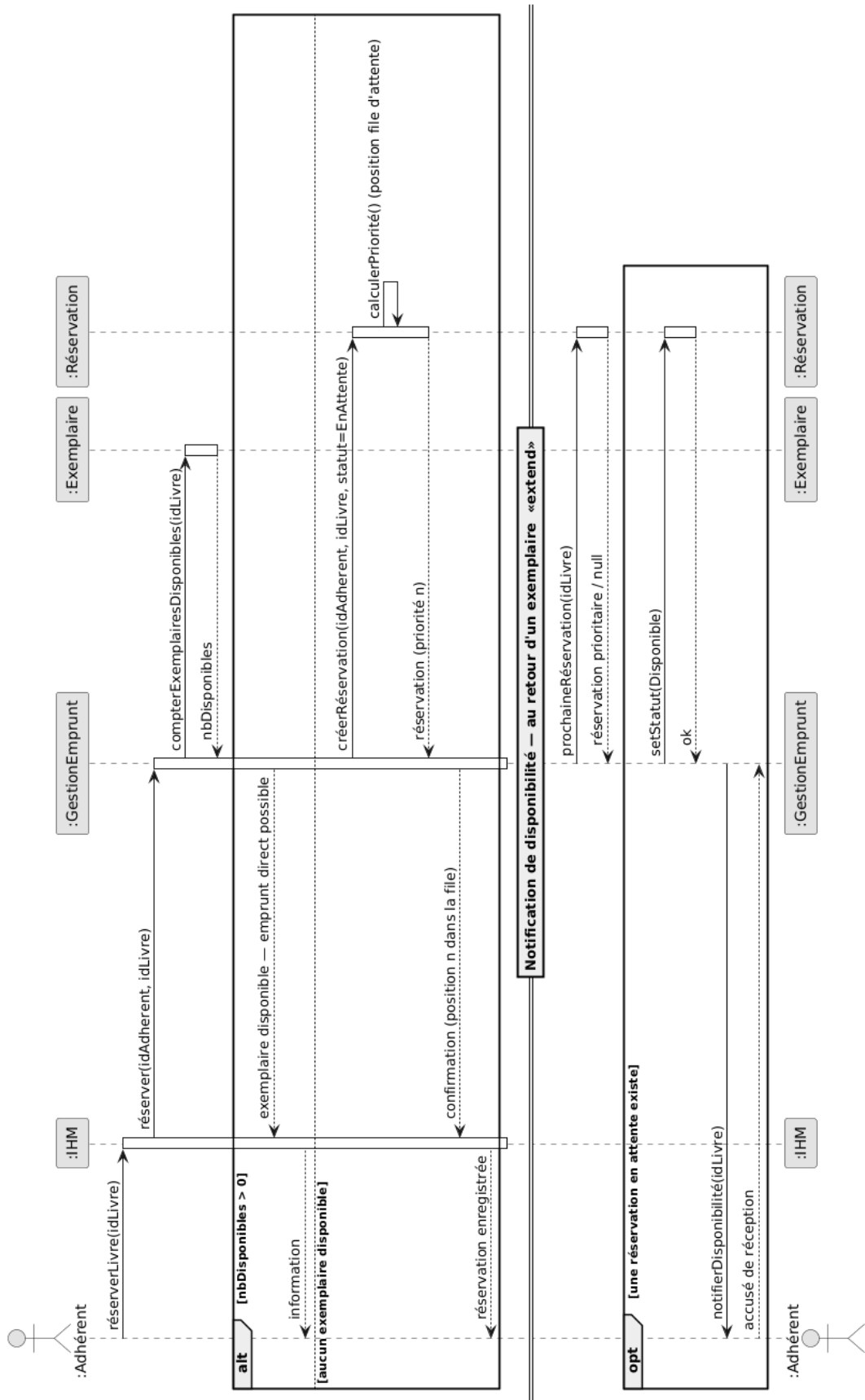


FIGURE 3.6 – Diagramme de séquence — Réserver un livre
Mise en file d'attente, calcul de la priorité et notification du réservataire prioritaire au retour.

4. Réalisation

Dans ce chapitre, nous décrivons la réalisation de l'application : nous justifions les choix technologiques, exposons l'architecture du code, illustrons les règles de gestion par des extraits de code, puis présentons les captures d'écran organisées par rôle.

4.1 Choix technologiques

L'application est développée en **Python 3** et n'utilise que la **bibliothèque standard** :

- **sqlite3** : moteur de base de données relationnel embarqué, stockant l'ensemble des données dans un unique fichier local (`bibliotheque.db`);
- **tkinter / ttk** : boîte à outils graphique native, pour une interface de bureau légère et soignée.

Ce choix répond directement à l'exigence « simple, graphique, sans dépendance externe » : aucune installation de bibliothèque tierce (ni *pip*, ni serveur, ni *framework*) n'est nécessaire. L'application est ainsi immédiatement exécutable sur tout poste disposant de Python, ce qui facilite son déploiement en contexte pédagogique.

4.2 Architecture du code

Le code est organisé en cinq modules aux responsabilités clairement séparées (architecture en couches), résumés dans le tableau 4.1.

TABLE 4.1 – Rôle des modules de l'application.

Module	Responsabilité
<code>database.py</code>	Connexion SQLite, création des tables et insertion des données de test.
<code>models.py</code>	Couche d'accès aux données (fonctions CRUD pour chaque entité).
<code>logique.py</code>	Règles de gestion (RG01–RG06) et workflows métier.
<code>interface.py</code>	Interface graphique Tkinter (connexion, menus par rôle, tableaux, formulaires).
<code>main.py</code>	Point d'entrée : initialise la base puis lance l'application.

Cette séparation (*données / accès / logique / présentation*) améliore la lisibilité et la maintenance : une règle métier se modifie dans `logique.py` sans toucher à l'interface, et un changement d'affichage n'impacte pas les règles.

4.3 Extraits de code clés

Le cœur métier réside dans `logique.py`. Le Listing 4.1 présente la fonction d'emprunt, qui enchaîne les contrôles RG04, RG01 et RG06 avant de créer l'emprunt.

```

1 def emprunter_livre(id_adherent, id_exemplaire):
2     adherent = models.obtenir_adherent(id_adherent)
3     if adherent is None:
4         return False, "Adherent introuvable."
5 
```

```

6     eligible, message = compte_eligible(adherent)      # RG04 : compte
      eligible ?
7     if not eligible:
8         return False, message
9
10    quota_ok, message = quota_respecte(adherent)        # RG01 : quota
      respecte ?
11    if not quota_ok:
12        return False, message
13
14    exemplaire = models.obtenir_exemplaire(id_exemplaire)
15    if exemplaire is None:
16        return False, "Exemplaire introuvable."
17    if exemplaire["disponible"] != 1:                    # RG06 : exemplaire
      disponible ?
18        return False, "Exemplaire indisponible."
19
20    date_emprunt = _aujourd'hui()
21    date_retour = _ajouter_jours(date_emprunt, DUREE_EMPRUNT_JOURS) #
      RG02 : +14 jours
22    id_emprunt = models.creer_emprunt(id_adherent, id_exemplaire,
23                                     date_emprunt, date_retour)
24    models.modifier_disponibilite_exemplaire(id_exemplaire, 0)
25    return True, "Emprunt n° " + str(id_emprunt) + " enregistré."

```

Listing 4.1 – Workflow d’emprunt appliquant RG04, RG01 et RG06.

Les deux contrôles d’éligibilité et de quota sont isolés dans des fonctions dédiées (Listing 4.2), ce qui rend la règle explicite et testable.

```

1 def compte_eligible(adherent):                          # RG04
2     if adherent["statut"] in ("Suspendu", "Bloque"):
3         return False, "Compte non eligible (statut)."
```

```

4     if models.compter_penalites_impayees(adherent["id_adherent"]) > 0:
5         return False, "Compte non eligible (penalite impayee)."
```

```

6     return True, ""
7
8 def quota_respecte(adherent):                            # RG01
9     nb = models.compter_emprunts_en_cours(adherent["id_adherent"])
10    if nb >= adherent["max_emprunts"]:
11        return False, "Quota atteint."
```

```

12    return True, ""

```

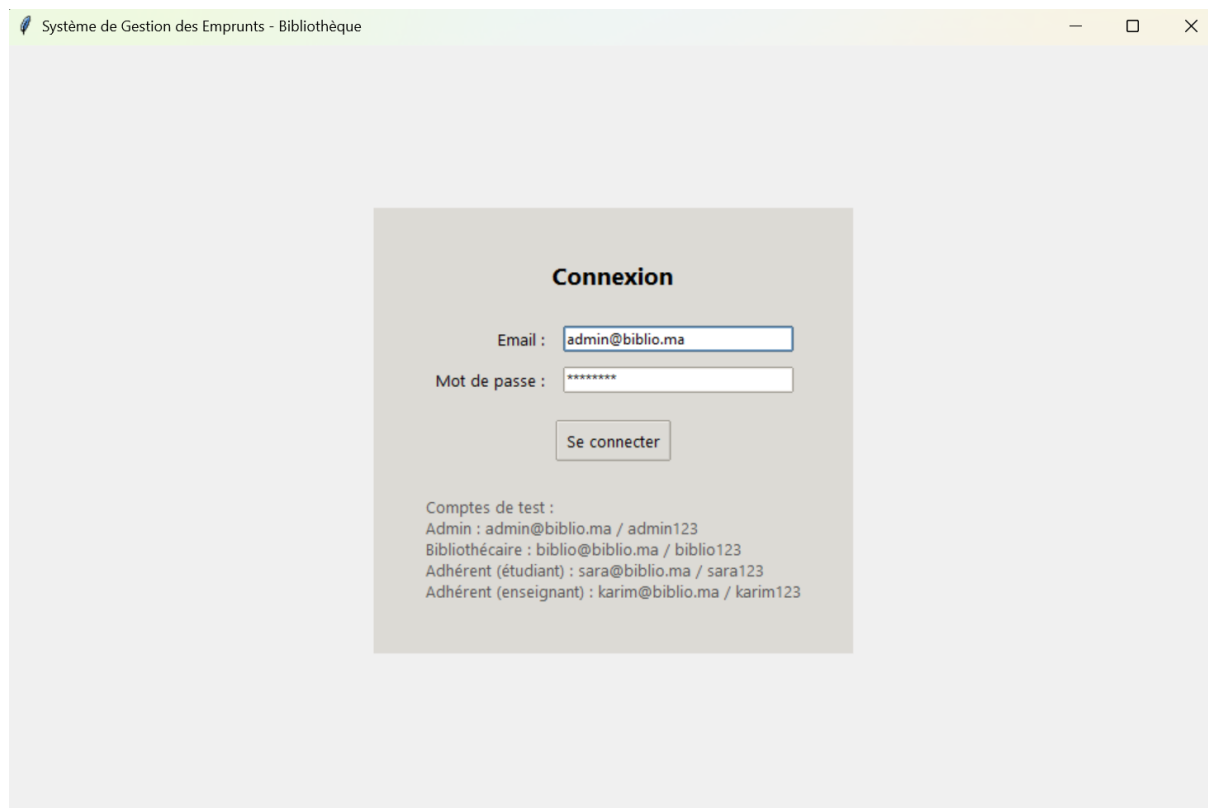
Listing 4.2 – Contrôles d’éligibilité (RG04) et de quota (RG01).

4.4 Captures d’écran de l’application

Cette section illustre l’application en exploitation, organisée par rôle. Chaque écran est présenté sur une page dédiée afin d’en préserver la lisibilité.

4.4.1 Accueil et authentification

L'écran d'accueil ci-dessous constitue le point d'entrée commun à tous les rôles.



Système de Gestion des Emprunts - Bibliothèque

Connexion

Email :

Mot de passe :

Comptes de test :

- Admin : admin@biblio.ma / admin123
- Bibliothécaire : biblio@biblio.ma / biblio123
- Adhérent (étudiant) : sara@biblio.ma / sara123
- Adhérent (enseignant) : karim@biblio.ma / karim123

FIGURE 4.1 – Écran d'authentification

Saisie de l'e-mail et du mot de passe ; l'orientation vers le menu dépend du rôle.

4.4.2 Espace Administrateur

L'administrateur gère les rôles et les emplacements, configure les quotas et consulte les indicateurs du tableau de bord.

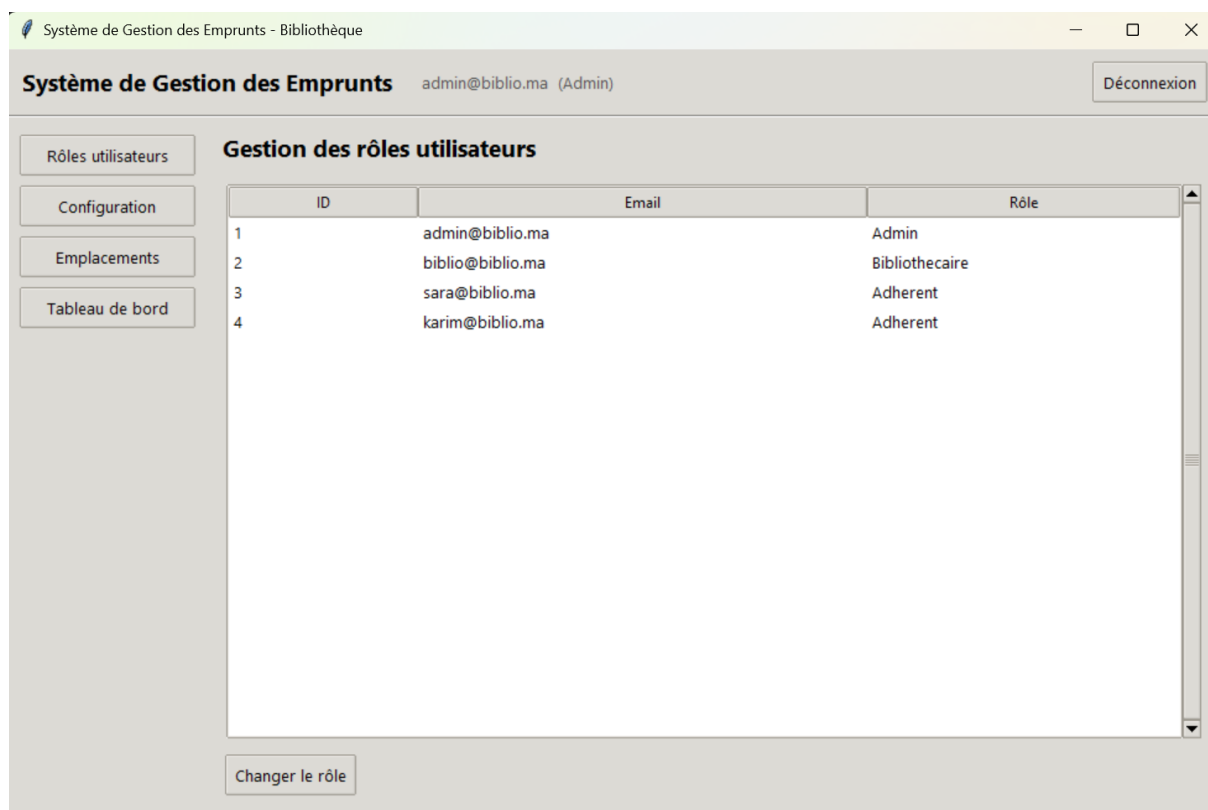
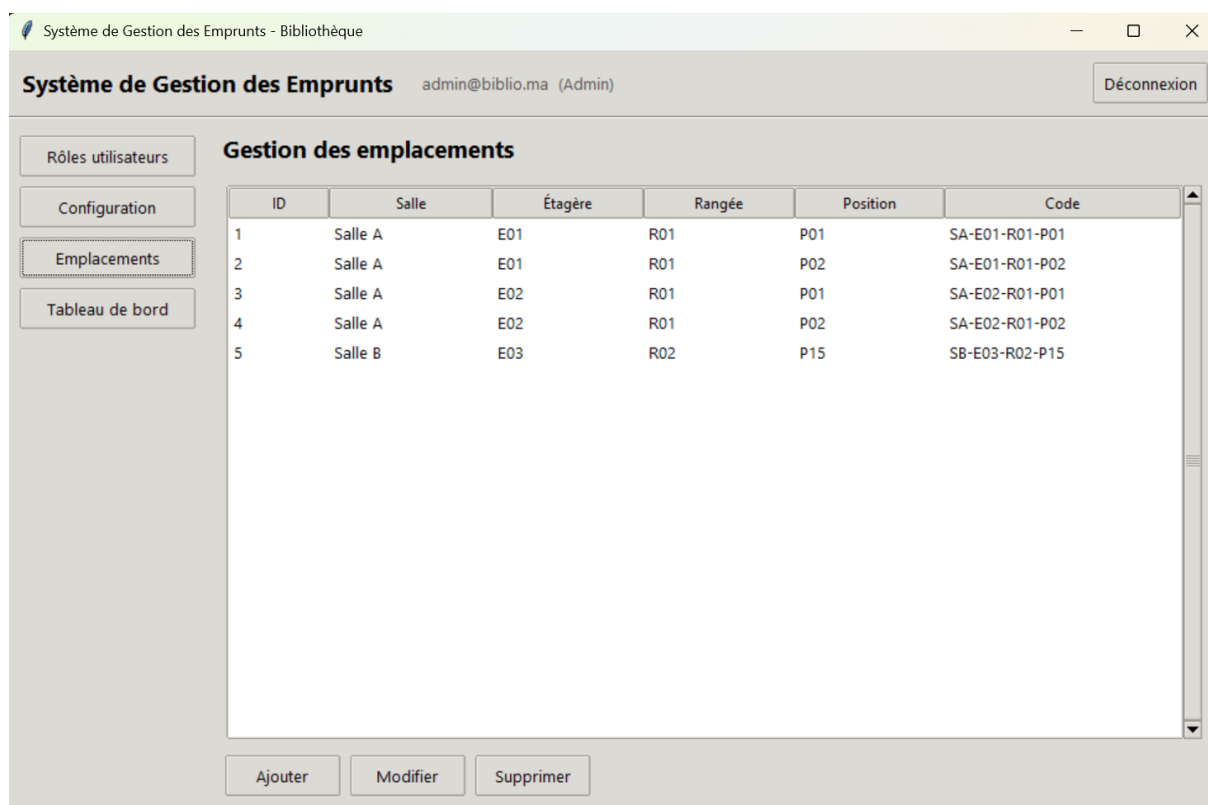


FIGURE 4.2 – Rôles utilisateurs

Liste des comptes et changement de rôle (Admin, Bibliothécaire, Adhérent).

**FIGURE 4.3 – Emplacements**

Emplacements physiques : salle, étagère, rangée, position et code unique (RG05).

The screenshot shows a web application interface for a library system. The title bar indicates 'Système de Gestion des Emprunts - Bibliothèque'. The main header displays 'Système de Gestion des Emprunts' and the user 'admin@biblio.ma (Admin)' with a 'Déconnexion' button. A sidebar on the left contains navigation buttons: 'Rôles utilisateurs', 'Configuration' (highlighted), 'Emplacements', and 'Tableau de bord'. The main content area is titled 'Configuration' and contains two sections. The first section, 'Quotas d'emprunts par adhérent (RG01)', features a table with the following data:

ID	Adhérent	Type	Quota
1	Alaoui Sara	Etudiant	3
2	Bennani Karim	Enseignant	5

Below the table is a 'Modifier le quota' button. The second section, 'Paramètres de gestion (fixes)', lists the following fixed parameters:

- Durée d'un emprunt : 14 jours (RG02)
- Prolongation : +7 jours, une seule fois (RG02)
- Pénalité de retard : 2 DH / jour (RG03)
- Présomption de perte : au-delà de 30 jours de retard (RG03)
- Frais administratifs (perte/vol) : 20 DH

FIGURE 4.4 – Configuration

Quotas d'emprunt par adhérent et paramètres de gestion (durée, prolongation, tarif de retard, frais administratifs).

**FIGURE 4.5 – Tableau de bord**

Livres les plus empruntés, emprunts en retard et nombre total de pertes/vols.

4.4.3 Espace Bibliothécaire

Le bibliothécaire dispose des écrans opérationnels couvrant le catalogue, les adhérents et le cycle emprunt/retour.

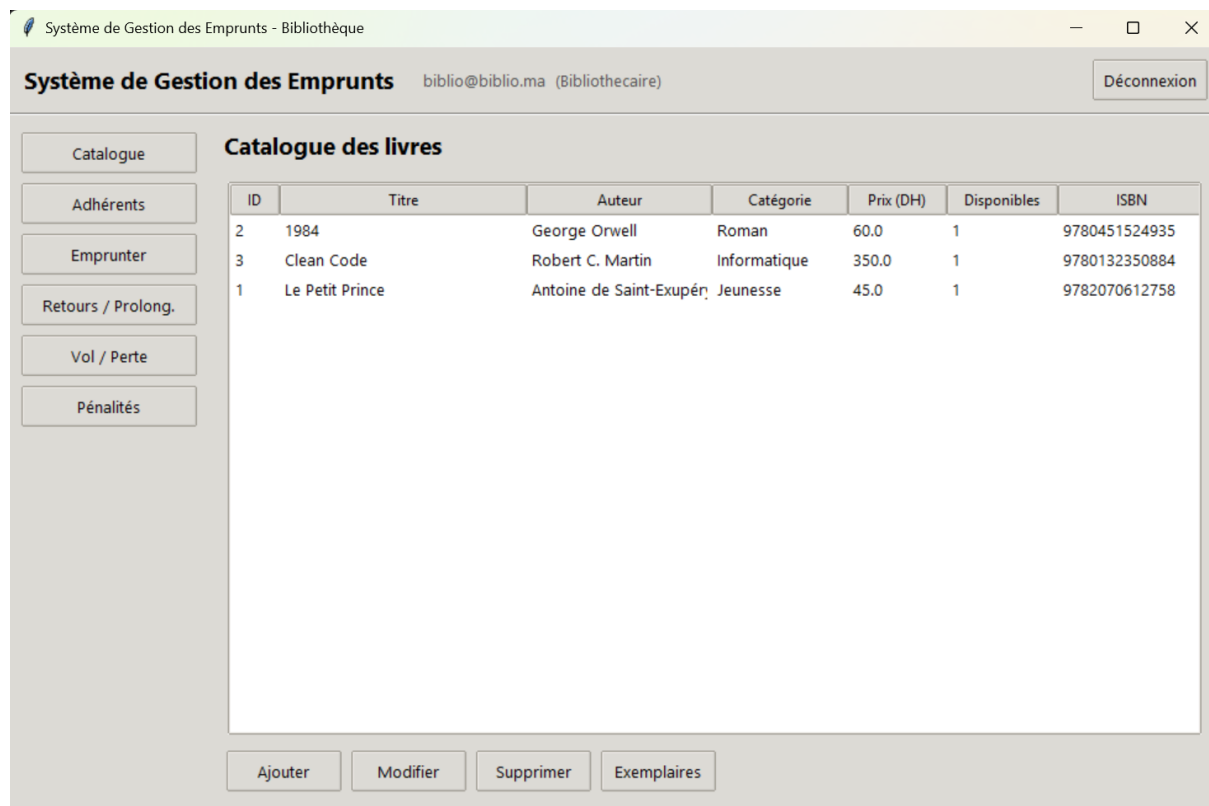
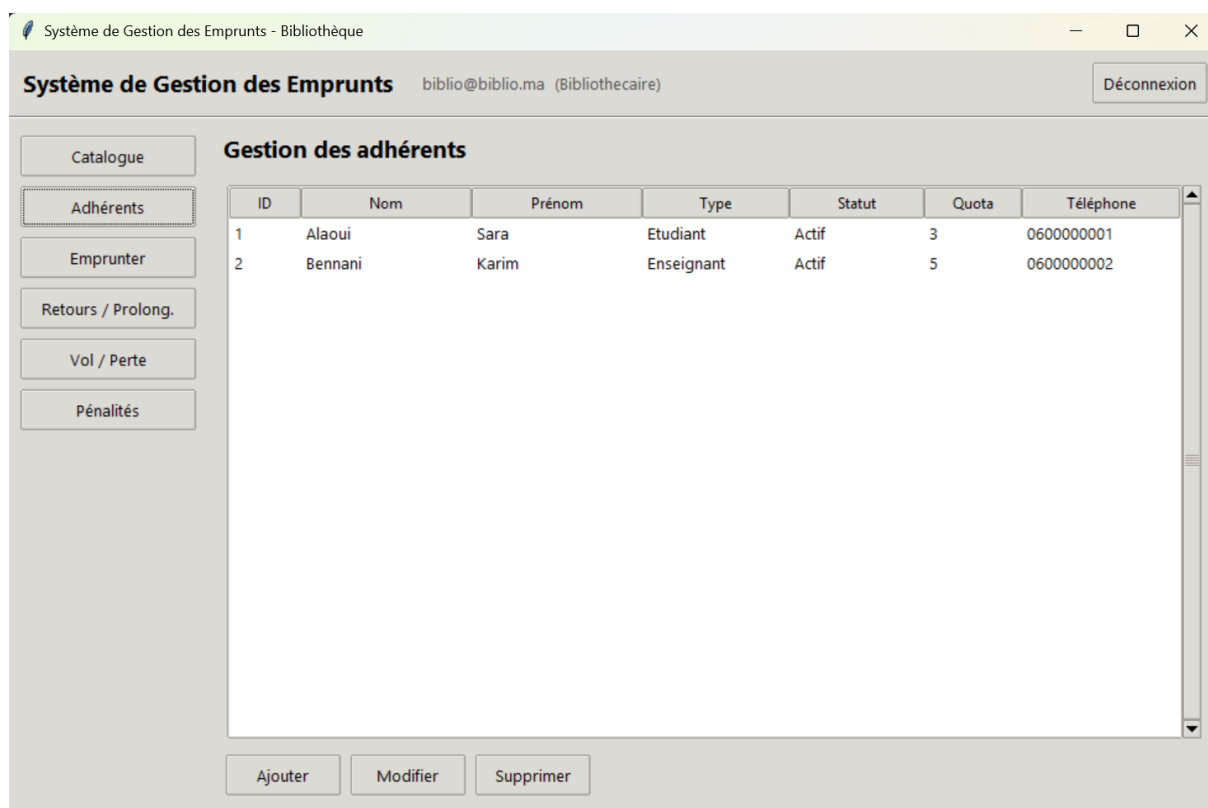


FIGURE 4.6 – Catalogue

Livres et nombre d'exemplaires disponibles ; CRUD et gestion des exemplaires.

**FIGURE 4.7 – Adhérents**

Gestion des adhérents : type, statut, quota et coordonnées.

Système de Gestion des Emprunts - Bibliothèque

Système de Gestion des Emprunts biblio@biblio.ma (Bibliothécaire) Déconnexion

Catalogue

Adhérents

Emprunter

Retours / Prolong.

Vol / Perte

Pénalités

Enregistrer un emprunt

Adhérent :

Exemplaire disponible :

Emprunter

FIGURE 4.8 – Emprunter

Choix d'un adhérent et d'un exemplaire disponible pour créer l'emprunt.

**FIGURE 4.9 – Retours / Prolongations**

Emprunts en cours : retour d'un exemplaire ou prolongation de +7 jours.

Système de Gestion des Emprunts - Bibliothèque

Système de Gestion des Emprunts biblio@biblio.ma (Bibliothécaire) Déconnexion

Catalogue

Adhérents

Emprunter

Retours / Prolong.

Vol / Perte

Pénalités

Gestion des pénalités

ID	Adhèrent	Livre	Type	Montant (DH)	Paiement	Date
----	----------	-------	------	--------------	----------	------

Payer Exonérer

FIGURE 4.10 – Pénalités

Paieement ou exonération d'une pénalité, avec déblocage automatique du compte.

Système de Gestion des Emprunts - Bibliothèque

Système de Gestion des Emprunts biblio@biblio.ma (Bibliothécaire) Déconnexion

Catalogue

Adhérents

Emprunter

Retours / Prolong.

Vol / Perte

Pénalités

Déclarer un vol ou une perte

N° emprunt	Adhérent	Livre	Code-barres	Statut
1	Alaoui Sara	Le Petit Prince	EX-0001	EnRetard
2	Bennani Karim	1984	EX-0003	EnCours

Déclarer Perte Déclarer Vol

FIGURE 4.11 – Vol / Perte

Déclaration d'une perte ou d'un vol : pénalité = prix + frais et blocage du compte.

4.4.4 Espace Adhérent

L'adhérent consulte le catalogue et son historique, et reçoit ses notifications de disponibilité. Les deux comptes de test (Sara, étudiante ; Karim, enseignant) illustrent ces écrans.

ID	Titre	Auteur	Catégorie	Prix (DH)	Disponibles	ISBN
2	1984	George Orwell	Roman	60.0	1	9780451524935
3	Clean Code	Robert C. Martin	Informatique	350.0	1	9780132350884
1	Le Petit Prince	Antoine de Saint-Exupéry	Jeunesse	45.0	1	9782070612758

FIGURE 4.12 – Catalogue (Karim)

Consultation et réservation par l'adhérent enseignant.

Système de Gestion des Emprunts - Bibliothèque

Système de Gestion des Emprunts sara@biblio.ma (Adherent) Déconnexion

Catalogue

Mon historique

Mes notifications

Catalogue des livres

ID	Titre	Auteur	Catégorie	Prix (DH)	Disponibles	ISBN
2	1984	George Orwell	Roman	60.0	1	9780451524935
3	Clean Code	Robert C. Martin	Informatique	350.0	1	9780132350884
1	Le Petit Prince	Antoine de Saint-Exupéry	Jeunesse	45.0	1	9782070612758

Réserver le livre

FIGURE 4.13 – Catalogue (Sara)

Consultation du catalogue par l'adhérente étudiante.

Système de Gestion des Emprunts - Bibliothèque

Système de Gestion des Emprunts karim@biblio.ma (Adherent) [Déconnexion](#)

[Catalogue](#) [Mon historique](#) [Mes notifications](#)

Mon historique

Mes emprunts

N°	Livre	Emprunté le	Retour prévu	Retourné le	Statut
2	1984	2026-06-12	2026-06-26	-	EnCours

Mes pénalités

ID	Livre	Type	Montant (DH)	Paieement
----	-------	------	--------------	-----------

FIGURE 4.14 – Historique (Karim)

Emprunts en cours et pénalités de l'enseignant.

Système de Gestion des Emprunts - Bibliothèque

Système de Gestion des Emprunts sara@biblio.ma (Adherent) Déconnexion

Catalogue

Mon historique

Mes emprunts

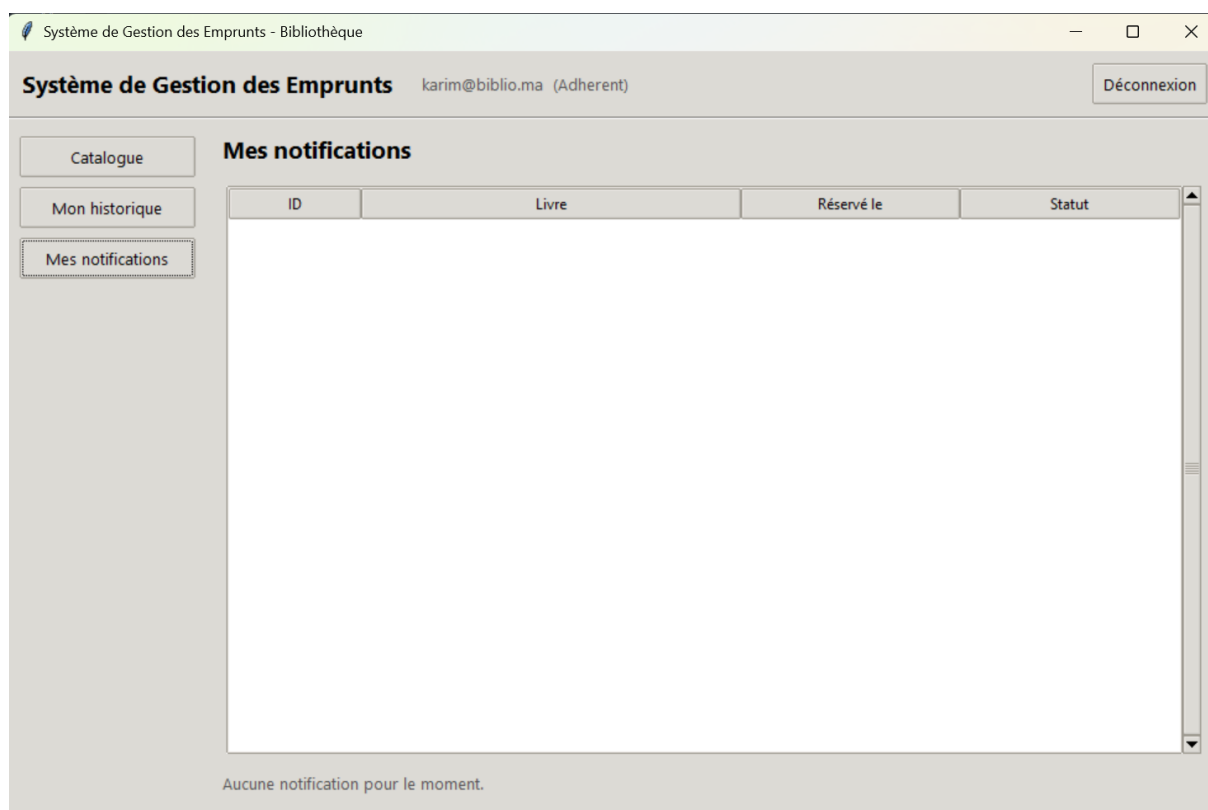
N°	Livre	Emprunté le	Retour prévu	Retourné le	Statut
1	Le Petit Prince	2026-05-28	2026-06-11	-	EnRetard

Mes pénalités

ID	Livre	Type	Montant (DH)	Paieement
----	-------	------	--------------	-----------

FIGURE 4.15 – Historique (Sara)

Historique de l'étudiante, incluant un emprunt en retard.

**FIGURE 4.16 – Notifications (Karim)**

Réservations devenues disponibles signalées à l'adhérent.

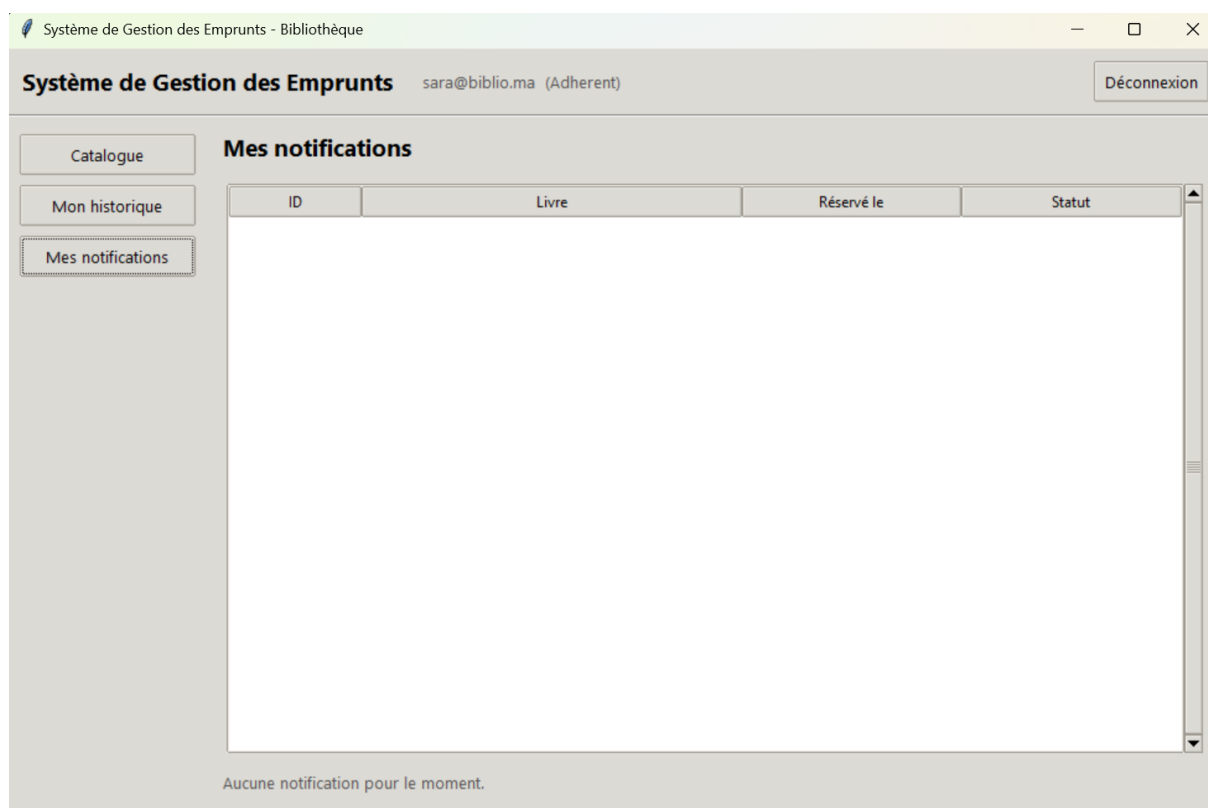


FIGURE 4.17 – Notifications (Sara)
Avis de disponibilité des livres réservés.

En conclusion, nous avons montré comment les choix techniques et l'organisation du code concrétisent les règles de gestion définies en amont ; le chapitre suivant dresse le bilan du projet.

5. Conclusion et perspectives

Dans ce dernier chapitre, nous dressons le bilan du travail réalisé et proposons des perspectives d'amélioration.

5.1 Bilan

Ce projet a permis de concevoir et de réaliser une application de bureau complète pour la gestion des emprunts d'une bibliothèque. La démarche Merise/UML a guidé l'analyse (acteurs, besoins, règles de gestion) et la conception (cas d'utilisation, classes, séquences), tandis que l'implémentation en Python/SQLite/Tkinter a concrétisé l'ensemble des fonctionnalités visées. Les trois exigences initiales sont satisfaites : la **traçabilité** grâce à l'emplacement unique de chaque exemplaire (RG05), la **sûreté** via la déclaration des vols/pertes et le blocage des comptes, et le **contrôle** par l'application automatique des quotas et des délais (RG01–RG04, RG06). L'architecture en couches assure une bonne lisibilité et facilite la maintenance.

5.2 Perspectives

Plusieurs évolutions permettraient d'enrichir la solution :

- **Version web** (architecture client/serveur) pour un accès multi-postes et une consultation à distance du catalogue ;
- **Notifications par e-mail** automatiques lors de la disponibilité d'une réservation ou à l'approche d'une échéance de retour ;
- **Statistiques avancées** et exports (PDF, tableurs) pour le pilotage : taux de rotation, saisonnalité des emprunts, indicateurs par catégorie ;
- **Sécurité renforcée** : hachage des mots de passe et journalisation des actions ;
- **Recherche et filtres** avancés dans le catalogue (par auteur, catégorie, ISBN).

Ces axes prolongeraient naturellement le travail réalisé tout en conservant la simplicité qui fait la force de la solution actuelle.

En définitive, nous avons mené ce projet de bout en bout, de l'analyse des besoins à la réalisation d'une application fonctionnelle, et nous avons identifié des pistes concrètes pour le faire évoluer.

Bibliographie

1. H. Tardieu, A. Rochfeld, R. Colletti. *La méthode Merise : Principes et outils*. Éditions d'Organisation.
2. P. Roques. *UML 2 par la pratique : études de cas et exercices corrigés*. Eyrolles.
3. M. Lutz. *Learning Python*. O'Reilly Media.
4. M. Owens. *The Definitive Guide to SQLite*. Apress.
5. A. Moore. *Python GUI Programming with Tkinter*. Packt Publishing.

Webographie

- Documentation officielle Python : <https://docs.python.org/3/>
- Module `sqlite3` (Python) :
<https://docs.python.org/3/library/sqlite3.html>
- Module `tkinter` (Python) :
<https://docs.python.org/3/library/tkinter.html>
- Site officiel de SQLite : <https://www.sqlite.org/docs.html>
- Documentation `ttk` (widgets thématiques) :
<https://docs.python.org/3/library/tkinter.ttk.html>